

2교시: .env.example과 secret 비노출

수업 목표

- .env.example과 실제 .env의 역할을 구분한다.
- 문서에 남겨도 되는 정보와 남기면 안 되는 값을 분리한다.
- secret masking 기준을 출력 예시로 확인한다.

개념 설명

.env.example은 팀원에게 필요한 key 이름과 형식을 알려주는 파일이다. 실제 password, token, cloud key를 담은 파일이 아니다. 반대로 .env는 로컬 실행을 위한 실제 값이 들어갈 수 있으므로 공개 repository, README, screenshot, 질문 게시글에 그대로 올리면 안 된다.

초보자는 실습용이니깐 괜찮다고 생각하기 쉽다. 하지만 습관이 중요하다. 수업용 password라도 그대로 캡처해 공유하는 방식에 익숙해지면, 이후 Docker Hub token, AWS access key, database password도 같은 방식으로 노출될 수 있다.

실행할 때는 .env를 자동으로 읽는다고 가정하지 않는다. docker run에서는 명시적으로 --env-file week2/day4/labs/env-report/.env를 붙여야 한다.

-e DB_PASSWORD=value처럼 명령줄에 값을 직접 넣으면 빠르지만 shell history와 화면 공유에 남기 쉽다. 여러 설정을 반복해서 쓸 때는 .env에 기록하고 --env-file로 로드하는 편이 낫다. 단, .env 자체를 공유하면 안 되고, 공유용으로는 .env.example만 둔다.

환경별 파일을 나눌 때도 같은 원칙이 적용된다. .env.dev, .env.staging, .env.prod처럼 파일을 나누는 것은 가능하지만, 운영 secret을 repository에 넣어도 된다는 뜻은 아니다. 수업에서는 구조를 이해하기 위해 값을 넣어 보지만, 실제 회사에서는 production password, API key, cloud credential은 Secret Manager, CI/CD secret, Kubernetes Secret 같은 별도 경로로 주입하는 것이 일반적이다.

공유할 수 있는 파일은 보통 값이 비어 있거나 placeholder만 있는 예시 파일이다.

```
# .env.prod.example
APP_ENV=prod
FEATURE_FLAG=off
API_BASE_URL=https://api.example.com
DB_PASSWORD=replace-me-securely
```

이 파일은 prod에는 어떤 key가 필요한가를 알려주는 문서 역할을 한다. 실제 .env.prod는 로컬 또는 배포 시스템 안에서만 관리한다.

실습 명령

```
cd /mnt/d/paperclip
sed -n '1,120p' week2/day4/labs/env-report/.env.example
cp week2/day4/labs/env-report/.env.example week2/day4/labs/env-report/.env
chmod +x week2/day4/labs/env-report/report.sh
docker run --rm --env-file week2/day4/labs/env-report/.env -v
"$PWD/week2/day4/labs/env-report:/work:ro" alpine:3.20 /work/report.sh
```

Expected:

```
APP_ENV=practice
FEATURE_FLAG=on
DB_PASSWORD=***masked***
```

문서화 기준

문서에 남겨도 되는 것	문서에 남기면 안 되는 것
DB_PASSWORD라는 key 이름	실제 password 값
--env-file .env 사용 방식	.env 전체 내용
DB_PASSWORD=***masked***	terminal에 찍힌 실제 token
.env.example	개인 .env
.env.prod.example	실제 .env.prod

환경별 secret 판단

파일	공유 여부	이유
.env.example	가능	key 이름과 placeholder만 포함
.env.dev	보통 로컬 전용	개인 개발값이 들어갈 수 있음
.env.staging	제한 공유	staging credential 포함 가능
.env.prod	공유 금지	운영 secret 포함 가능

실패 예시

```
DB_PASSWORD=my-real-password
```

이 출력이 README나 screenshot에 남으면 실패다. Day 4의 목표는 secret manager를 깊게 다루는 것이 아니라, 로컬 Docker 실습에서도 값을 그대로 남기지 않는 습관을 만드는 것이다.

Kubernetes와 Terraform에서도 같은 습관이 이어진다. Kubernetes manifest에는 ConfigMap으로 공개 가능한 설정을 두고, password/token은 Secret으로 분리한다. Terraform에서는 *.tfvars에 환경별 값을 나눌 수 있지만, 민감한 값은 state와 repository 노출 위험까지 함께 판단해야 한다.

다음 연결

다음 교시는 container를 실행하고 logs와 HTTP 응답을 분리해 정상 여부를 판단한다.